

Git Branching Commands Lab

Push a Python grade calculator to GitHub from VS Code, then master every branching command on the same code.

Part 1: The Python Code

Create a folder named **branching-lab** and open it in VS Code (File > Open Folder). Create a new file named **grades.py** and type the following code:

```
# grades.py
# A simple student grade calculator

def calculate_grade(marks):
    if marks >= 90:
        return "A"
    elif marks >= 75:
        return "B"
    elif marks >= 60:
        return "C"
    else:
        return "F"

if __name__ == "__main__":
    marks = 82
    print("Marks:", marks)
    print("Grade:", calculate_grade(marks))
```

Run it once in the VS Code terminal (Terminal > New Terminal):

```
python grades.py

Marks: 82
Grade: B
```

Part 2: Push the Code to GitHub

Type these commands in order in the VS Code terminal. Note: branch names are case-sensitive — always use lowercase **main**.

```
git init
git add grades.py
git commit -m "Add grade calculator"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/branching-lab.git
git push -u origin main
```

Refresh the GitHub page. grades.py is now online. The setup is complete.

Part 3: Branching Commands — One by One

1. git branch — list branches

```
git branch
* main
```

The asterisk (*) marks the branch you are currently on.

2. git branch <name> — create a branch

```
git branch feature-average
```

List again. The new branch exists, but you are still on main:

```
git branch
    feature-average
* main
```

3. git switch — move to a branch

```
git switch feature-average
Switched to branch 'feature-average'
```

On this branch, add a new function to grades.py below calculate_grade():

```
def average(marks_list):
    return sum(marks_list) / len(marks_list)
```

Save, then commit the change on this branch:

```
git add grades.py
git commit -m "Add average function"
```

4. git switch main — see that branches are isolated

```
git switch main
```

Open grades.py. The average function is **gone** — it exists only on feature-average. Switch back and it reappears:

```
git switch feature-average
```

This isolation is the whole purpose of branching.

5. git switch -c — create and switch in one command

```
git switch main
git switch -c feature-topper
Switched to a new branch 'feature-topper'
```

On this branch, add another function:

```
def topper(marks_dict):
    return max(marks_dict, key=marks_dict.get)
```

Commit it:

```
git add grades.py
git commit -m "Add topper function"
```

Note: **git checkout -b** is the older command that does the same thing.

6. git branch -v — see the last commit on each branch

```
git branch -v
    feature-average a1b2c3d Add average function
    feature-topper e4f5g6h Add topper function
* main            9f8e7d6 Add grade calculator
```

7. git merge — bring a branch into main

```
git switch main  
git merge feature-average
```

Open `grades.py` — the `average` function is now on `main`. Merge the second branch too:

```
git merge feature-topper
```

8. git log --graph — visualize the branch history

```
git log --oneline --graph --all
```

The graph shows where branches split from main and where they merged back.

9. git push — send branches to GitHub

Push the updated main, then push a feature branch itself:

```
git push
git switch feature-topper
git push -u origin feature-topper
```

On GitHub, use the branch dropdown (left of the file list) to switch between main and feature-topper online.

10. git branch -m — rename a branch

```
git branch -m feature-average feature-avg
```

List branches to confirm the new name:

```
git branch
```

11. git branch -d — delete a merged branch

Both feature branches are merged, so they are safe to delete:

```
git branch -d feature-avg
git branch -d feature-topper

Deleted branch feature-avg (was a1b2c3d).
Deleted branch feature-topper (was e4f5g6h).
```

Delete the branch on GitHub too:

```
git push origin --delete feature-topper
```

12. git branch -D — force delete an unmerged branch

Create a throwaway branch, commit a change, and try to delete it without merging:

```
git switch -c experiment
# add any comment line to grades.py, save
git add grades.py
git commit -m "Experimental change"
git switch main
git branch -d experiment

error: the branch 'experiment' is not fully merged
```

Git protects unmerged work. Force delete only when you are sure the work is not needed:

```
git branch -D experiment
```

13. Bonus: create a merge conflict and resolve it

Create two branches from main. On **branch-a**, change the marks value to 95 and commit. Switch to main, create **branch-b**, change the same line to 55 and commit. Then:

```
git switch main
git merge branch-a
git merge branch-b

CONFLICT (content): Merge conflict in grades.py
Automatic merge failed; fix conflicts and then commit the result.
```

VS Code highlights the conflict with **Accept Current Change / Accept Incoming Change** buttons. Choose one, save, then complete the merge:

```
git add grades.py
git commit -m "Resolve marks conflict"
```

Branching cycle summary: git switch -c <branch> → edit and commit → git switch main → git merge <branch> → git branch -d <branch> → git push.